ChatGPT

30-Day Branching-Chat MVP Roadmap

Architecture & Core APIs

- **Workspaces:** Manage separate chat workspaces. E.g. `POST /workspaces` (create), `GET /workspaces/:id` (info). Each workspace holds its conversation tree.
- **Nodes (Messages):** Store each user/assistant message. Endpoints include `POST /nodes` (add message) and `GET /workspaces/:id/tree` (fetch the full message tree). To fetch a linear branch context, use `GET /branches/:id/nodes` which returns messages from root through the branch head.
- **Branches:** Enable forking. Endpoints include `POST /branches/fork` (split at a node), `POST /branches/:id/activate` (switch active branch), and `PATCH/DELETE /branches/:id` (rename or archive). Each branch record stores `workspace_id`, `fork_node_id`, `head_node_id`, `name`, and chosen `model`.
- **Chat Streaming:** A streaming chat endpoint (e.g. `POST /chat/stream`) accepts `{messages, model, branchId}` and streams back tokens (via SSE or WebSockets) using the chosen LLM. Each complete assistant response is saved as a new node.
- **Token Budget API:** Track context size with `GET /branches/:id/context-budget`, returning used vs available tokens for that branch.
- **Export & Merge (Optional):** `GET /export/branch/:id?format=json|md` and `GET /export/workspace/:id?format=...` for downloading conversations. `POST /merge` (if implemented) to insert content from one branch into another.

Database Schema (PostgreSQL):

- `workspaces` (id, user_id, created_at).
- `branches` (id, workspace_id, fork_node_id, head_node_id, name, model, archived_flag).
- `nodes` (id, workspace_id, branch_id, parent_id, role [user/assistant], content, content_hash, token_count, created_at).
- `merge_records` (id, src_branch_id, dst_branch_id, injected_node_id).
- `snapshots` (id, workspace_id, snapshot_hash, data).

Minimum Viable Scope

- **Must-haves:** User auth; basic chat with one model (e.g. GPT-4); **branch creation** from any message; visual **conversation tree** (React Flow); context budget management (token counting, basic summarization). These core features form the “smallest slice” of the vision [1].
- **Should-haves:** Multi-model support (add Claude), export (JSON/Markdown), a simple landing page.
- **Could-haves:** Basic merge (concatenate branch content), minimal onboarding (sample data).
- **Won’t do (v1):** Highlight-&-branch UI, advanced merges (diff/summarize between branches), real-time collaboration. All extras go to “v2” if time. We fix the deadline (30 days) and flex scope [2].

Team Roles (3 Devs + PM)

- **Frontend Developer (Dev A):** Builds the React UI and Canvas (React Flow) for the chat and tree. Creates components for chat input, message nodes, branch sidebar, etc.
- **Backend Developer (Dev B):** Implements the Fastify/Node API and database schema. Builds endpoints for workspaces, messages, branches, export. Handles data modeling (Postgres, Redis) and security checks.
- **Full-stack/LLM & DevOps (Dev C):** Handles LLM integration (streaming chat logic, context assembly, summarization). Also sets up CI/CD pipelines, AWS deployment (ECS, RDS), monitoring and staging/production environments.
- **Product Manager:** Guides priorities, removes blockers, and handles demos/QA. In early sprints may pair on code reviews or helper tasks (e.g. writing test scripts, prep for demos).

All devs pair or swap roles as needed. Early sprints pair FE+LLM to validate UI/streaming and BE+DevOps for infra. Later, cross-functional pairing ensures vertical slices. Daily standups and weekly demos keep everyone aligned (continuous delivery via CI/CD [3]).

Detailed 30-Day Plan

Week 1 (Days 1–7): Foundation & Chat MVP

Goal: Build one complete vertical slice: user can sign up, send a message to GPT-4, and see the response appear on a visual tree.

- **Day 1 – Infrastructure & Init:**
- *DevOps/Dev C:* Create AWS account, IAM roles. Set up AWS ECR and ECS (Fargate) with a "hello world" container. Configure GitHub repo with CI/CD (GitHub Actions).
- *Backend (Dev B):* Initialize Fastify+TypeScript project. Add logging (pino), CORS, input validation. Write Dockerfile; configure Docker Compose for local Postgres+Redis.
- *Frontend (Dev A):* Scaffold React (Vite or Next.js) + TypeScript. Install React Flow, Zustand (or Redux). Create an empty chat page with a React Flow canvas.
- *LLM Integration (Dev C):* Integrate OpenAI (ai-sdk). Write a test script calling GPT-4 via stream API to verify tokens are returned.

- *Deliverables:* Backend "Hello" API deployed; blank React app running locally; streaming GPT-4 client test.

- **Day 2 – Database & Auth:**

- *DevOps/Dev C:* Provision AWS RDS (Postgres) and ElastiCache (Redis). Store DB credentials in AWS Parameter Store.
- *Backend (Dev B):* Define DB schema (Drizzle ORM): tables for workspaces, branches, nodes, merge_records, snapshots. Enable `ltree` if needed for path queries. Run migrations on RDS.
- *Frontend (Dev A):* Integrate Clerk (or Auth0) for auth. Wrap app in `<ClerkProvider>`. Create login/signup pages using prebuilt UI components. Protect main route (redirect to login if unauthenticated). Add `<UserButton>` in header.
- *Backend (Dev B):* Add Clerk Fastify plugin to validate JWT, extract userId. Add auth middleware to secure endpoints.

- *QA:* Test sign-up/login → user redirected to a new workspace page. DB tables populated with initial data.

- **Day 3 – Core API & Chat Endpoint:**

- *Backend (Dev B):* Build core REST APIs:
  - `POST /workspaces` – create a new workspace for the user.
  - `GET /workspaces/:id` – fetch workspace info.
  - `GET /workspaces/:id/tree` – return all nodes for the workspace (initially root only).
  - `POST /nodes` – create a new message node (with parent_id, role).
- *Backend (Dev B):* Implement `POST /chat/stream` (SSE): accept `{messages, model, branchId}`, call `ai.streamText()`, pipe tokens to client. On end, save the assistant's response as a new node. Handle client disconnects gracefully.
- *Frontend (Dev A):* Build chat input UI (textarea + send button). On send, call `/chat/stream` and render incoming tokens inside a new React Flow node.
- *Frontend (Dev A):* Create a **MessageNode** component: shows role (user/assistant), content (markdown), timestamp.

- *QA:* Sign up, create workspace, send a chat message. Verify backend logs and DB show the user node and the assistant node; confirm response streams in real time on UI.

- **Day 4 – Canvas UI & Integration:**

- *Frontend (Dev A):* Integrate React Flow with a dagre layout. After each message, add a node and edge to the graph and re-layout. Handle variable node heights by measuring first, then applying dagre.
- *Frontend (Dev A):* Style the canvas (e.g. scrollable area, basic node styling). Add a loading indicator while waiting.
- *Backend (Dev B):* Write a recursive CTE query to fetch the ancestor chain for a given branch head.
- *Integration (Dev C):* Connect front-end actions to backend: when user submits, `POST /nodes` then re-fetch `GET /tree`. When assistant responds, append to state.
- *DevOps (Dev C):* Add Sentry (frontend and backend) and CloudWatch logs.

- *QA:* Ensure a new message appears on canvas as a node after sending and receiving.

- **Day 5 – End-to-End Test & Buffer:**

- *All:* Perform full flow: sign up, log in, create a workspace, send a message, see GPT-4 reply on canvas. Log any bugs.
- *DevOps:* Deploy backend and frontend to a staging environment (ECS) via CI/CD. Request ACM certificate and set up domain alias.
- *PM:* Document any open issues. Prepare list of next tasks.

- *Buffer Tasks (if ahead):*

  - *Frontend:* Polish message node UI; add auto-scroll on new node; animate layout changes.
  - *Backend:* Add workspace list page; implement pagination for large message trees.
  - *LLM Dev:* Add retry logic for streaming failures (3x attempts).

- **Days 6–7 – Weekend Buffer:**

- Use extra time to catch up on any incomplete tasks or to improve existing features.
- Milestone: **Vertical Slice Complete:** Deployed staging app where a logged-in user can chat with GPT-4 and see the conversation build as a tree on a canvas. (Meets core MVP functionality [2].)

Week 2 (Days 8–14): Branching Core Features

Goal: Implement forking so users can branch off at any message, with independent contexts and visualized tree.

- **Day 8 – Branch Data Model & API:**
- *Backend:* Create `branches` table (id, workspace_id, fork_node_id, head_node_id, name, model, archived).
- *Backend:* `POST /branches/fork` – input `{workspaceId, forkNodeId, name}`; creates new branch pointing at that node. Initialize `head_node_id = forkNodeId`. Return branch ID.
- *Backend:* `GET /branches/:id/nodes` – return the linear ancestor path from root up to the branch's current head (collect via parent links).
- *Frontend:* Add a context menu on message nodes (right-click or "…" menu) with "Branch from here." Clicking opens a modal to name the branch (and optionally pick a model).
- *Frontend:* On submit, call `POST /branches/fork`, then refresh the canvas.
- *LLM Dev:* Modify chat endpoint to accept a `branchId`. When sending, traverse that branch's ancestor chain (not the mainline) to assemble prompt context. Mark new nodes with branch ID.

- *Specification:* Document how the UI indicates the active branch (e.g. color or a label) and how to switch.

- **Day 9 – Branch Visualization:**

- *Frontend:* Update React Flow layout to handle branches: children of the fork node should appear on a new branch path. Differentiate active branch (bright color) vs inactive (faded). Animate transitions.
- *Backend:* Implement `POST /branches/:id/activate` (no body). Store activeBranchId in workspace or user session. `GET /workspaces/:id/full-tree` returns all nodes with branch metadata.
- *Frontend:* "Continue in Branch": when a branch is active, sending a message posts with that branch's `head_node_id` as parent. Branch sidebar highlights the branch.

- *Backend:* Ensure branch context isolation: the `head_node_id` for each branch is tracked and used for context. (If a branch's fork point is edited, treat nodes as immutable to avoid cross-effects.)

- **Day 10 – Branch Management & Performance:**

- *Frontend:* Build a branch sidebar/panel: list all branches with name, fork node snippet, last activity. Click to switch. Include "Rename" and "Archive" actions. Add a minimap overview of large trees.
- *Backend:* `PATCH /branches/:id` (rename), `DELETE /branches/:id` (mark archived). `GET /workspaces/:id/branches` returns branch metadata (count, timestamps). Use Redis caching (`tree:${workspaceId}`) for performance.
- *Backend:* Implement content-addressable hashing for deduplication: on node creation, compute `contentHash = SHA-256(role + content + parentId)` and use `ON CONFLICT DO NOTHING`.

- *Performance Test:* Spin up a test workspace with ~10 branches × 50 messages. Measure: tree load time, layout time, context assembly time. Identify any slow queries or UI lags.

- **Day 11 – Chat-in-Branch Polish:**

- *Frontend:* Add a "Focused Chat View": when actively chatting in a branch, allow toggling a linear chat pane alongside the tree. This avoids overwhelming the user with the entire tree. Auto-scroll on new messages.
- *DevOps:* Add rate limiting on LLM endpoints (e.g. 20 req/min/user) via Redis token bucket. Prevent duplicate submits using the contentHash key (lock SET NX).
- *LLM Dev:* Implement token counting: use a fast tokenizer to compute each node's tokens and store `token_count`.
- *Backend:* Add `GET /branches/:id/token-count` to sum ancestor tokens and calculate usage vs model limit.

- *Frontend:* Display a token budget bar (green/yellow/red) below the input showing "used/available". Tooltip with breakdown (recent vs summarized tokens).

- **Day 12 – Branch QA & Bug Fixes:**

- *All:* Thoroughly test branching: create 3+ branches from the same root, add messages in each, switch and verify context. Ensure no cross-branch leakage. Check that tree view updates correctly.
- *All:* Fix critical bugs from testing (wrong context, broken UI, streaming issues in branches).
- *DevOps:* Deploy fixes to staging and confirm stability.

- *Buffer (if time):*

    - *Frontend:* Add keyboard shortcuts (e.g. Cmd+B to branch, Cmd+Shift+M to merge mode).
    - *Backend:* Optimize recursive CTE with materialized path (Postgres ltree) if needed.

- **Days 13–14 – Weekend Buffer:**

- If branching features are solid, start auxiliary tasks: implement Claude adapter (stub), export JSON/MD UI, or refine data model. If not, use this as catch-up time.

Milestone (End of Week 2): Fully functional branching. Users can fork from any message, chat separately in branches, and navigate the tree. The core product value is realized.

Week 3 (Days 15–21): Multi-Model, Context Budget & Polish

Goal: Add secondary model, context management, export, and finalize MVP polish.

- **Day 15 – Multi-Model Support:**
- *LLM Dev:* Integrate Anthropic Claude (e.g. Sonnet-4) via the AI SDK. Normalize API differences so streaming works.
- *Backend:* Extend `branches` schema to include a default `model`. Update chat endpoint to accept/record model choice.
- *Frontend:* Add a model selector dropdown on the chat input or branch creation modal. Indicate model on each assistant node (icon or label).
- *Backend:* Implement `PATCH /branches/:id/model` to change a branch's model.

- *Test:* Verify GPT-4 vs Claude responses side by side. Ensure token counting works (different tokenizers).

- **Day 16 – Context Budget & Summarization:**

- *LLM Dev:* Implement context trimming: if `total_tokens` for a branch >80% of the model limit, automatically summarize older nodes. Approach: keep last 10 messages verbatim, summarize earlier history with a prompt ("Condense the conversation history into key points."), save summary as special node. Cache summaries in Redis by hash of node range.
- *Backend:* Add model context windows (e.g. 128k for GPT-4, 200k for Claude). Provide `GET /branches/:id/context-budget` returning `{used, available, needsSummarization}`.
- *Frontend:* Show context budget progress bar (green/yellow/red). Tooltip details breakdown. If summarization occurs, indicate it (e.g. an icon on the tree).

- *Test:* Simulate deep branches (50+ messages). Verify summarization triggers appropriately and LLM responses remain coherent.

- **Day 17 – Export & Workspace Features:**

- *Frontend:* Add "Export" menu. Options: "Export Branch" (JSON or Markdown) and "Export Workspace" (full JSON).
- *Backend:*
  - `GET /export/branch/:id?format=json|md`: JSON returns node-tree; Markdown returns linear conversation (e.g. `## User\n{content}\n\n## Assistant\n{content}`).
  - `GET /export/workspace/:id?format=json`: all branches.
- *Backend:* Implement snapshots: `POST /snapshots` to create a snapshot hash (SHA-256 of ordered node hashes) for versioning/determinism. `GET /snapshots/:hash` to fetch.
- *QA:* Test exports contain correct data. Verify JSON can be re-imported (if supporting reimport).

- *PM:* Test workspace flow (create/rename/delete). Ensure a graceful message if no branches exist.

- **Day 18 – Merge (Optional) & Landing Page Start:**

- *Merge (if time):* Basic "concat" merge: `POST /merge` taking source branch nodes and appending them into target branch as a summarized note. Track in `merge_records`. Simple UI to multi-select nodes and pick target branch. If not on track, skip this.
- *Frontend/DevOps:* Draft landing page in separate branch: outline hero, problem/solution, 3 feature cards (Branch, Multi-model, Export), sign-up CTA. Use Tailwind CSS or a template for speed.

- *Backend:* Cache entire tree structure per workspace in Redis (cache-aside: update on changes, TTL 5m). This improves navigation responsiveness.

- **Day 19 – Polishing & Hardening:**

- *Frontend:* UI polish: loading skeletons on data fetch, user feedback toasts (e.g. "Branch created", "Export successful"). Improve mobile responsiveness.
- *Backend:* Strengthen API: uniform error format, catch edge cases (missing resources). Review SQL queries for injection safety. Ensure all routes require auth and user can only access their own data.
- *LLM Dev:* Robust error handling: exponential backoff on timeouts (3 retries), fallback to alternative model if primary fails, show user-friendly error ("Model busy, try again").

- *DevOps:* Conduct an automated regression test (could be a script) covering all flows from sign-up to branch creation. Log results.

- *QA:* Fix any high-priority bugs discovered.

- **Days 20–21 – Weekend Buffer:**

- If ahead: continue landing page (implement hero, embed demo GIF, add links).
- If behind: finish any unfinished tasks from Week 3 (e.g. export polish, small bug fixes).

Milestone (End of Week 3): MVP is feature-complete. Chat with GPT-4/Claude works across branches, context is managed, exports exist, and the UI is stable. Any remaining items are minor.

Week 4 (Days 22–30): Stabilization & Launch

Goal: Freeze scope, optimize, test under load, prepare for public launch.

- **Day 22 – Feature Freeze & Landing Page**:
- **Feature Freeze:** No new features (anything beyond bugs goes to a "v2 backlog"). This enforces fixed time, variable scope [2].
- *Frontend:* Finalize landing page: add screenshots (or placeholders), demo GIF, clear CTAs. Implement sign-up redirect on CTA.
- *Backend:* Optimize DB: run `EXPLAIN ANALYZE` on key queries (ancestor CTE, tree fetch); add missing indexes (e.g. on `(workspace_id, branch_id)`). Tune Postgres connection pool.
- *DevOps:* Configure production ECS service (separate from staging). Set autoscaling policies (min 1, max 4 tasks, CPU target ~70%). Migrate config to prod environment variables.

- *DevOps:* Configure custom domain (Route 53) and apply SSL via AWS ACM. Test HTTPS.

- **Day 23 – Security & Compliance:**

- *Backend:* Security audit: verify every API checks JWT and ownership (user can't access others' branches). Sanitize inputs to prevent XSS/SQL injection. Configure CORS to only allow the prod domain.
- *Frontend:* Accessibility pass: ensure buttons/menus have aria-labels, keyboard focus works (tabbing through nodes), color contrast meets WCAG. Responsive checks on tablet.
- *DevOps:* Rate-limit production API (e.g. API Gateway throttling or nginx limits) to guard against abuse.
- *Legal:* Prepare Privacy Policy and Terms of Service (use an online generator). Link them in footer.

- *Support:* Create a Discord server or Slack community link, set up a support email.

- **Day 24 – Load Testing & Monitoring:**

- *All:* Perform load test (e.g. with k6 or Artillery) simulating ~50 concurrent users: actions include sign-up, branching chats, streaming.
- *Backend:* Identify bottlenecks (e.g. slow context assembly, DB locks). Potential fixes: add Redis caching for hotspot queries, batch DB writes (e.g. buffer tokens). Ensure Redis cache hit rate is high.
- *Frontend:* Test React Flow with 100+ nodes. If UI lags, consider lazy-loading off-screen branches or using a virtual scroller.

- *DevOps:* Set up monitoring dashboards:
    - **CloudWatch:** alarms on CPU >80%, Memory >85%, 5xx rate >5%.
    - **Sentry:** ensure production DSNs active; alerts on exceptions.
    - **Analytics:** finalize PostHog (or Plausible) to track events (sign_up, first_message, create_branch, model_switch, export).
- **Day 25 – Full QA:**
- *All:* Each team member tests on different browser/device. Use a shared bug log to record issues. Classify bugs as P0 (blocks launch), P1 (urgent fix), P2 (cosmetic).
- *DevOps:* Verify logging and monitoring flows (alerts triggered on simulated error).
- *PM:* Ensure every P0 is addressed. Any unresolved P0 feature may be disabled or demoted.
- **Day 26 – Critical Fixes & Analytics:**
- *All:* Fix all P0 bugs. Aim to resolve quickly; if a P0 is complex, consider rolling back that feature for launch.
- *DevOps:* Instrument analytics on landing page (page views, sign-ups) and in-app events. Verify data is coming in (test event firing).
- *Frontend:* Last UI tweaks; finalize demo video (60s showing sign-up → branch).
- **Day 27 – Pre-Launch Review:**
- *All:* Conduct final smoke test on **production**. Verify auth keys, DB connections, LLM keys, and domain.
- *PM:* Check off critical flows (see checklist below).
- *Marketing:* Prepare Product Hunt assets (logo, tagline, 5 screenshots, maker comments). Prepare Hacker News “Show HN” draft.
- *DevOps:* Document rollback plan (e.g. ECS can rollback to previous task definition; keep old container tag handy).
- **Day 28 – Launch Prep:**
- *PM/DevOps:* Finalize launch checklists and responsibilities (war-room plan). Ensure on-call channels are set.
- *Frontend:* Post meta tags (Open Graph) for social sharing. Ensure landing page looks good on social preview.
- *All:* Announce “soft launch” to internal testers. Fix any last-minute critical bug.
- **Day 29 – Soft Launch (Friends & QA):**
- Invite ~20 testers (team members, friends). Monitor logs (Sentry) and metrics closely. Collect feedback. Patch critical issues immediately. Prepare final “release candidate.”
- **Day 30 – Public Launch:**

- **Morning (12:01 AM PST):** Launch on Product Hunt ("BranchChat: Git-like branching for AI conversations"), post to Hacker News and Twitter thread (with demo video).
- **Throughout Day:** Monitor all systems. Respond to user feedback in PH, HN, social. Deploy any urgent fixes.
- Celebrate – the MVP is live and doing what it promised: enabling multi-threaded AI chats with full context, just as envisioned.

Continuous Integration & Deployment

- **CI/CD Pipeline:** Use GitHub Actions. On each commit to `main`, run lint/tests, build Docker images, and push to ECR. On merges to `main`, auto-deploy to staging or (after sign-off) to production via AWS ECS. This maintains the MVP in a "deployable" state at all times [3].
- **Infrastructure-as-Code:** (Optional) Terraform or CloudFormation to provision AWS ECS, RDS, IAM roles, etc. Ensure environments (staging vs prod) are separate.
- **Rollback Strategy:** ECS supports versioned deployments; keep the last stable image so you can revert on high error rates. Maintain scripts to quickly switch traffic (e.g. via Route 53 or ECS traffic shifting).

Monitoring & Observability

- **Error Tracking:** Sentry (frontend + backend) to catch exceptions or promise rejections.
- **Metrics & Logs:** AWS CloudWatch for infrastructure (CPU, memory), API Gateway/ALB metrics (5xx count), and custom logs. Set alarms for anomalies (e.g. >5% 5xx [4]).
- **Usage Analytics:** PostHog or Plausible to track user events (sign-ups, first branch, model switches). Use this to measure KPIs (Branch Adoption Rate, Session Depth, token savings).
- **Dashboards:** CloudWatch Dashboard showing key metrics (error rate, active tasks, CPU, etc.). PostHunt/Maker channels for alerts.
- **Escalation Plan:** Define who responds to alerts (e.g. DevOps covers infra, BE covers API issues, FE covers UI issues). Keep a "runbook" in Notion.

Risks, Mitigations & Scope Cuts

- **Scope Creep:** *High likelihood/impact.* Mitigation: Enforce feature freeze (Day 22) and strictly separate "must-have" vs "nice-to-have." Adhere to Lean Startup principle – focus on validated core [1] [2].
- **UI Performance:** Large branch trees could make React Flow sluggish. Mitigation: Limit DOM nodes (lazy-load collapsed branches), optimize layout (incremental updates), or cap displayed nodes (page deep threads). If unresolved, communicate it as a known issue.
- **LLM API Limits/Outages:** High impact. Mitigation: Implement retries/backoff from Day 1, set conservative rate limits per user. Have a fallback (e.g. switch to Claude if GPT-4 fails). Cache frequent prompts to reduce calls.
- **Context Assembly Bugs:** Critical. Mitigation: Write integration tests for branch queries. Mark nodes immutable. Use content-hash deduplication to detect anomalies (no duplicate content in different branches) [5].
- **Team Availability:** If someone gets ill, devs should be cross-trained. Use daily standups to share progress. **Pair program** key components so no single point of failure.
- **Security / Data Privacy:** Mitigation: Encrypt data at rest (RDS encryption). Ensure JWT/session security. Keep API keys in secrets manager. Regular code reviews of auth logic.
- **Time Overruns:** Buffer ~20% time in each sprint. Prioritize critical path tasks (backend DB & branching logic) early. If deadlines slip, drop "should-have" features first (multi-model could be postponed to v2, for example).

- **Load/Scale:** Mitigation: Use autoscaling on ECS (min=1, max=4). For extreme loads, consider horizontal DB read replicas (out of scope for MVP).
- **Legal/Compliance:** If dealing with user data, ensure logs are ephemeral. Mitigation: Explicit terms/privacy policies and minimal data collection (only chat content, no PII).

"Fail fast, fail cheap. Better to know in 30 days than 12 months." – By shipping the core branchable chat MVP quickly, we minimize wasted effort and can learn from real users [6].

Each day's plan is tightly scoped. By end of Day 30, the public MVP will be live, tested, and monitored, with a clean deployment process and rollback plan in place.

Key Checklist Before Launch:

- [ ] User signup and auth works end-to-end.
- [ ] Chat with GPT-4 (and Claude) works in all branches.
- [ ] Branch UI: create, list, switch, archive.
- [ ] Context budget prevents unseen token overrun.
- [ ] Export (JSON/MD) functional.
- [ ] All P0 bugs resolved; no data-loss bugs.
- [ ] Onboarding flow guides new users to first branch within 2 min.
- [ ] CI/CD pipeline green (staging → prod).
- [ ] ECS service healthy (1–4 tasks).
- [ ] SSL, domain, load balancer configured.
- [ ] Sentry and CloudWatch alerts set (CPU, memory, 5xx).
- [ ] Legal pages and support channels live.
- [ ] Launch assets ready (PH listing, HN post, demo video).

By following this aggressive, week-by-week roadmap and "cut scope, not corners" approach [2], the team can deliver a functional branching chat workspace to users in 30 days, then iterate from real feedback.

Sources: We adopt agile/MVP best practices [1] [2] [3] and AWS deployment patterns to guide this plan.

---

[1] [2] [3] [5] [6] How to Build Your MVP in 30 Days: The Complete 2026 Guide with AI Acceleration | Startupbricks Blog
https://www.startupbricks.in/blog/mvp-in-30-days

[4] Recommended alarms - Amazon CloudWatch
https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Best_Practice_Recommended_Alarms_AWS_Services.html